

Three.js: 101

Presentation

Three.js est un moteur 3D pour navigateur web, écrit en javascript.

Il a été conçue pour être simple à utiliser et performant, beaucoup de sites l'utilisent pour afficher des animations élégantes sur leurs pages web et des jeux vidéo.

Il exploite WebGL et le DOM, ce n'est donc utilisable que dans le contexte d'un navigateur, pas dans NodeJS.

Attention, Three.js n'est pas un moteur de **jeu**, il s'occupe uniquement du rendu d'images 3D, à l'inverse de moteurs comme Unreal Engine ou Unity, qui gèrent aussi tout ce dont un jeu peut avoir besoin (physique, collisions, audio, networking, ...).

Supports d'information

ThreeJS

Vous trouverez toutes les informations nécessaire à l'apprentissage et l'utilisation de cette librairie sur son site officiel:

<https://threejs.org/>

Sur cette page, vous trouverez des exemples de sites où elle a été utilisée.

Dans le menu de gauche, dans la partie **Learn** vous trouverez notamment:

- Des **exemples** des techniques habituelles avec le code associé
- La **documentation**, séparée en deux parties:
 - **docs** La référence de toutes les classes et méthodes de la librairie
 - **manual** Le guide complet pour apprendre à utiliser la librairie

Introduction à la 3D par Mozilla

Mozilla a écrit un guide rapide pour apprendre les bases théoriques de la 3D:

[3D on the web](#)

Getting started

Il existe différentes façons d'intégrer Three.js à une page web.

La plus simple, si vous n'avez besoin que d'afficher votre rendu ThreeJS, est d'utiliser une simple page HTML et de compiler votre programme avec [Vite](#).

Si vous avez des besoins plus complexes, notamment intégrer le rendu à une page web existante, ou créer des menus ou d'autres composants 2D, vous devrez intégrer le moteur dans un projet classique type Next ou Vue.

Prérequis

Vous devez avoir installé NodeJS sur votre ordinateur.

<https://nodejs.org/fr>

Starter-kit de base (Vite)

La solution la plus simple est d'utiliser Vite pour compiler votre programme ThreeJS.

Vous pouvez partir de ce projet de base: [boilerplate-three-vite.zip](#)

Décompressez cette archive dans votre ordinateur, puis ouvrez le dossier dans votre éditeur de code.

Installez les dépendances avec `npm i` et lancez le projet avec `npm start`.

Vous aurez un serveur web démarré qui rechargera automatiquement le code à chaque changement. Ouvrez votre navigateur à la page indiquée pour voir la page web.

Composition du programme

- `index.html` : Notre fichier de base, qui est vide mais sert à charger notre script principal. Le moteur de rendu prendra tout l'espace de la page.
- `src/main.js` : Le code javascript minimal pour afficher un cube qui se déplace.

Attention: Plus votre programme deviendra complexe, plus il sera nécessaire de découper le programme en plusieurs fichiers, et il faudra idéalement utiliser des classes.

Starter-kit avec Next.js

Si vous préférez intégrer Three.js dans une page web plus complexe, il est possible de l'utiliser.js avec Next.js, en utilisant les [ref de React](#) pour donner l'élément `canvas` à Three.js, ce qui nous permet de le placer où on le souhaite dans la page.

Vous pouvez partir de ce projet de base: [boilerplate-three-next.zip](#)

De même, installez les dépendances avec `pnpm i` et lancez le projet avec `pnpm dev`.

Composition du programme

- `app/page.tsx` : La page principale qui va décider de l'emplacement du rendu de notre jeu.
- `app/three-app.tsx` : Un composant `client` qui va charger notre jeu et le placer dans le DOM de manière responsive, c'est à dire que le rendu se redimensionnera automatiquement si la fenêtre du navigateur change de taille.
- `game/index.ts` : Notre moteur de jeu qui initialisera Three.js et notre scène.

SSR ?

Attention, three.js est un programme javascript **uniquement** destiné aux navigateurs (il utilise WebGL, pas disponible dans NodeJS)

Si vous l'utilisez dans un framework front tel que Next.js, vous ne pourrez l'intégrer que dans des composants **client-side** (`"use client"`).

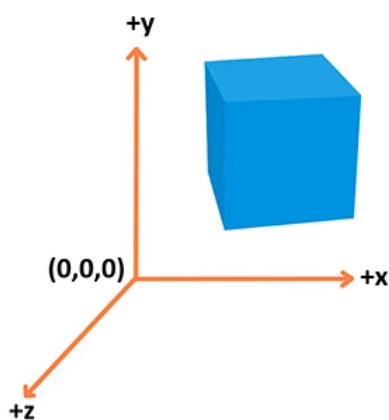
Concepts

Voici les termes clés utilisés par ThreeJS. Chacun de ces termes sont reliés à des classes Javascript dans la librairie, souvent un implémenté sous différentes formes

Vector

<https://threejs.org/docs/index.html?q=vector#api/en/math/Vector3>

Un **Vector** est un ensemble de valeurs du même type, pour la plupart du temps des coordonnées, et sur lequel on pourra effectuer des calculs récurrents (addition, multiplication, norme, ...). Possède les attributs `x`, `y`, `z`



```
const vector = new THREE.Vector3(1, 1, 1); // Vecteur
vector.x = 2;
vector.y = 3;
vector.z = 4;
```

jsx

Geometry

<https://threejs.org/docs/index.html#api/en/geometries/BoxGeometry>

Une **Geometry** représente la forme que prendra un objet 3D. Cela pourra être un cube, une sphere ou une forme plus complexe. Elle est constituée d'un ensemble de points reliés entre eux, qui forment des polygone, qui eux même forment des surfaces.

```
const geometry = new THREE.BoxGeometry(1, 1, 1); // Cube
const geometry = new THREE.SphereGeometry( 15, 32, 16 ); // Sphere
```

c

Material

<https://threejs.org/docs/index.html?q=mater#api/en/materials/MeshBasicMaterial>

Un **Material** sert à décrire comment apparaît une surface: sa couleur, sa texture et surtout comment elle réagira à la lumière.

```
const material = new THREE.MeshStandardMaterial({ color: 0x00ff00 });
```

c

Mesh

<https://threejs.org/docs/index.html?q=mesh#api/en/objects/Mesh>

Un **Mesh** est la combinaison d'une **Geometry** et d'un **Material**. C'est lui qu'on pourra placer dans notre scène.

```
const mesh = new THREE.Mesh(geometry, material);
```

c

Object3d

La plupart des objets de la scène héritent de la classe **Object3D**, notamment **Mesh**

<https://threejs.org/docs/index.html?q=Object#api/en/core/Object3D>

Chaque objet possède une multitude d'attributs et de fonctions pour les modifier, notamment:

- **position** (Vector3) : permet de déplacer l'objet
- **rotation** (Vector3) : permet de faire tourner l'objet

```
mesh.position.x = 2;  
mesh.rotation.y += Math.PI/2; // Un quart de tour
```

jsx

Rappel sur les angles

En threejs, les angles sont exprimés en **Radians**.

$360^\circ = 2 * \pi$ radians

Groupes

Un **Object3D** peut contenir d'autres **Object3D** (descendants), ce qui permet de créer des groupes et sous groupes d'objets. Grâce à cela, déplacer l'objet parent fera déplacer de la même façon tous ses descendants.

```
const person = new THREE.Object3D();  
person.add(head);  
person.add(legs);
```

js

```
person.position.x += 2; // déplacera la tête et les pieds.  
console.log(person.children); // 2 objets
```

Light

<https://threejs.org/docs/?q=light#api/en/lights/Light>

La lumière est un élément fondamental de la scène, car c'est elle qui éclairera l'ensemble des objets, sans elle on ne les verrait pas.

Il existe différents types de lumière:

- **Ambiant**: éclaire partout sans ombres
- **Point**: Eclaire partout autour d'elle telle une ampoule
- **Directional**: Eclaire dans une seule direction
- ... etc, voir la doc

```
const light = new THREE.AmbientLight( 0x404040 ); // soft white light  
  
const light = new THREE.PointLight( 0xff0000, 1, 100 );  
light.position.set( 50, 50, 50 );  
  
const directionalLight = new THREE.DirectionalLight( 0xffffff, 0.5 );  
light.target = targetObject;  
  
scene.add( light );
```

js

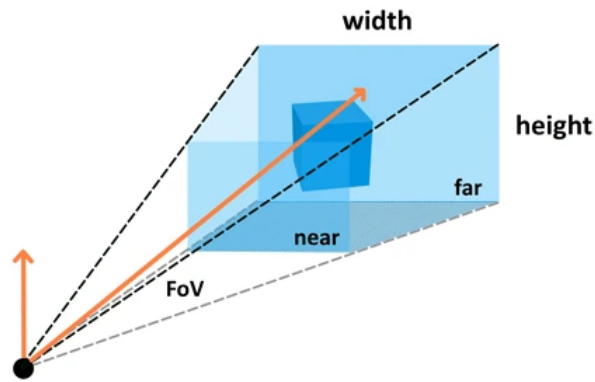
Camera

<https://threejs.org/docs/index.html?q=came#api/en/cameras/PerspectiveCamera>

La caméra sera notre point de vue sur la scène, et déterminera donc ce que l'on verra à l'écran. Elle possèdera un champ de vision, et tout ce qui ne se trouvera pas dans celui-ci ne sera pas affiché.

```
const camera = new THREE.PerspectiveCamera(  
  75, // Field of view  
  window.innerWidth / window.innerHeight, // aspect ratio  
  0.1, // near distance  
  1000 // far distance  
);
```

c



Scene

<https://threejs.org/docs/index.html?q=scene#api/en/scenes/Scene>

Une scene est un ensemble d'objets que l'on va vouloir afficher à l'écran.

```
const scene = new THREE.Scene();  
scene.add(mesh); // Ajout de notre cube a la scene
```

c

Renderer

<https://threejs.org/docs/index.html?q=ren#api/en/renderers/WebGLRenderer>

Le renderer est le moteur de rendu. C'est lui qui se chargera d'assimiler toutes les informations à afficher et les transformer en une image 2D à afficher à l'écran.

Le renderer threejs va creer un element HTML de type `canvas` qu'il faudra ajouter au DOM

Nous devons demander au renderer de faire un rendu a chaque fois que l'on veut une nouvelle image, et celui ci prendra en entrée notre **scene** et notre **camera**.

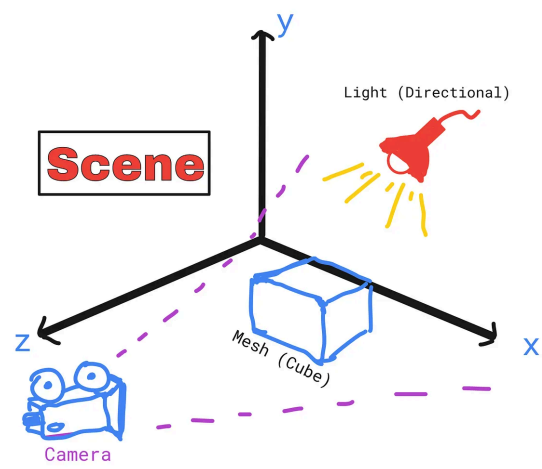
```
const renderer = new THREE.WebGLRenderer();  
renderer.setSize(window.innerWidth, window.innerHeight);  
document.body.appendChild(renderer.domElement);  
  
renderer.render(scene, camera);
```

c

Important

Pour afficher quelque chose à l'écran, il faut au minimum un **Mesh** (composé d'une **Geometry** et d'un **Material**), une **Camera**, une **Light**, une **Scene** et un **Renderer**.

Schema d'une scene



Animer la scene

Pour faire évoluer notre scene, nous allons transformer nos objets présents (translation, rotation, échelle...) à notre souhait

La boucle d'animation

La fonction `render.render(scene, camera);` effectue un rendu de la scène du point de vue de la caméra, mais ne le fait qu'une seule fois. Si on déplace les objets dans la scène sans rappeler cette fonction, rien ne se passera à l'écran.

Pour que notre scène affichée évolue, il va falloir appeler cette fonction pour chaque image rendue.

Il n'est pas possible d'appeler nous même cette fonction de rendu quand on le désire. Seulement le navigateur peut vous proposer de faire cela, via [requestAnimationFrame](#). Cela permet d'adapter la fréquence de rendu à la machine qui va executer votre programme (60 images par secondes sur la plupart des appareils), et aussi d'économiser des ressources quand c'est possible (le rendu ne sera pas effectué si l'onglet est caché par exemple).

La fonction `setAnimationLoop` sert à indiquer quelle fonction appelée dès que le navigateur peut afficher une nouvelle image.

```
function animate() {  
  cube.position.x += 0.01;  
  cube.rotation.y += 0.01;  
  
  render.render(scene, camera);  
}  
  
render.setAnimationLoop(animate);
```

js

Ici on décide de déplacer et tourner le cube de 0.01 à chaque image (normalement 0.6 par seconde à 60fps).

Etant donné que nous ne connaissons pas à l'avance la fréquence de rafraichissement au moment du rendu, et que cela peut évoluer, si on veut un déplacement constant il faut savoir combien de temps s'est écoulé depuis le dernier rendu. On peut faire ça manuellement en utilisant `Date.now()`, mais la fonction `animate` prend aussi un paramètre qui indique quand s'est effectué le dernier rendu (par rapport au démarrage de l'application).

```
let lastRenderTime = 0;  
function animate(renderTime) {  
  const delta = (renderTime - lastRenderTime) / 1000; // conversion en secondes  
  cube.position.x += 1 * delta;  
  lastRenderTime = renderTime;  
}
```

js

Ainsi, on peut garantir que le cube se déplacera de manière constante peut importe les fluctuations de rendu (1 par seconde).

Pour un usage plus avancé, on pourra également utiliser la classe [Clock](#)

Interactivité

Une page web avant tout

Comme vous le voyez pour le renderer, nous restons dans une page web et donc toutes les interactions que nous pouvons avoir en Javascript avec le DOM (`document`) sont toujours disponibles (evenements tels que clavier, souris, ...)

Clavier

Vous pouvez écouter les touches du clavier appuyés par l'utilisateur en utilisant la fonction `addEventListener` du DOM

```
document.addEventListener("keydown", (e) => { // touche appuyée
  switch (e.code) {
    case "KeyW": {
      // ...
      break;
    }
    case "KeyS": {
      // ...
      break;
    }
    case "KeyA": {
      // ...
      break;
    }
    case "KeyD": {
      // ...
      break;
    }
  }
});

document.addEventListener("keyup", (e) => { // touche relachée
  /// ...
});
```

jsx

Retrouvez les differents codes clavier ici: <https://www.toptal.com/developers/keycode>

Souris

```
document.addEventListener("mousedown", (e) => { // clic appuyé
  if (e.button == 0) { // clic gauche

  } else if (e.button == 2) { // click droit

  }
});

document.addEventListener("wheel", (e) => { // molette
  if (e.wheelDelta > 0) {

  } else if (e.wheelDelta < 0) {

  }
});
```

Gamepad

Il est possible d'utiliser des manettes de jeu avec l'API GamePad.

La documentation est disponible ici:

https://developer.mozilla.org/fr/docs/Web/API/Gamepad_API/Using_the_Gamepad_API

A l'inverse du clavier et de la souris, il n'est pas possible de déclencher un événement lors d'un appui sur une touche, nous devons alors manuellement faire du polling pour lire l'état de la manette.

```
function poll() {
  const gamepads = navigator.getGamepads();
  for(const gamepad of gamepads) {
    console.log(gamepad.axes);
    console.log(gamepad.buttons);
  }
}
```

Pour connaître quelles touches/axes correspondent à votre manette vous pouvez utiliser les sites suivants:

- <https://hardwaretester.com/gamepad>
- <https://luser.github.io/gamepadtest/>

Imports

Il est possible d'importer des ressources externes dans votre projet, tels que des modèles 3D ou textures existants.

Vous trouverez plein de modèles 3D gratuit en ligne notamment sur sketchfab:

<https://sketchfab.com/3d-models/popular>

Charger des modèles

Téléchargez de preference les modèles sous le format `gltf`

```
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";

const modelLoader = new GLTFLoader();
const gltf = await modelLoader.loadAsync("/assets/models/hovercar/scene.gltf");
scene.add(gltf.scene);
```

jsx

Charger des textures

```
const textureLoader = new THREE.TextureLoader();
const texture = await textureLoader.loadAsync("/assets/textures/block/dirt.png")

const material = new THREE.MeshBasicMaterial({ map: texture });
```

jsx

Distribution des assets

Les modeles et textures sont chargés au moment de l'execution du code par le client, pas par le serveur ou par le compilateur. Il est donc necessaire que les ressources soient disponibles par le navigateur.

En general, (avec Vite et Next), vous pouvez placer les assets dans le dossier `public`, qui rendra les fichiers qui sont dedans accessibles à l'url `/`.

Notions utiles

Skybox

La skybox permet d'afficher une image de fond dans le monde affiché pour donner une impression d'un univers (ciel, ville...) sans avoir besoin de modèles 3D.

Télécharger une image de fond au format Equirectangulaire

```
const textureLoader = new THREE.TextureLoader();
const texture = await textureLoader.loadAsync("./textures/skymap.png");
```

jsx

```
texture.mapping = THREE.EquirectangularReflectionMapping;
texture.colorSpace = THREE.SRGBColorSpace;
scene.background = texture;
```

Animation des modèles

Certains modèles 3D possèdent une animation pour les faire bouger dans le temps. Pour cela, il est nécessaire d'utiliser un **AnimationMixer** et de le mettre à jour à chaque frame.

```
const mixer = new THREE.AnimationMixer(gltf.scene);
const clip = THREE.AnimationClip.findByName(gltf.animations, "animation.steve.w
const action = mixer.clipAction(clip);

// Dans la boucle d'animation
mixer.update(delta);
```

Controls

Three.js integre différents Controls qui permettent un déplacement de la caméra selon le type de gameplay voulu. Par exemple, **PointerLockControls** est idéal pour les jeux types FPS, **OrbitControls** pour du TPS.

Pointer Lock

```
import { PointerLockControls } from "three/addons/controls/PointerLockControls.js";
const controls = new PointerLockControls(camera, document.body);
document.body.addEventListener("click", function () {
  controls.lock();
});
```

Detection d'objets

Raycaster

Le Raycaster permet de détecter des objets se trouvant dans l'axe d'un rayon. Ce rayon part d'un point, et possède une direction.

Cela peut être très utile pour détecter des collisions ou savoir sur quel est l'objet visé par l'utilisateur par exemple.

```
let raycaster = new THREE.Raycaster();

// Définir le rayon sur la position/direction de la camera
raycaster.setFromCamera( new Vector2(0, 0), camera );
```

```
// definir origine/direction manuellement
raycaster.set(origin, direction);

// Detecter les collisions avec une liste d'objets
const intersects = raycaster.intersectObjects( objectsList, true ); // true ind
```

Intersection

Pour detecter une superposition entre deux objets, on peut utiliser des Box.

Cet objet prend la forme d'un cube, et il est très facile de determiner si il y a une intersection entre deux cubes.

Ce cube est une approximation de notre modèle mais il permet d'avoir un calcul de collision rapide.

```
const boundingBox = new THREE.Box3();
boundingBox.setFromObject(object);

const boundingBox2 = new THREE.Box3();
boundingBox2.setFromObject(object2);

if(boundingBox.intersectsBox(boundingBox2)) {
  //...
}
```

jsx

Mise a jour de la taille de fenetre

Vous remarquerez que de base, si vous redimensionnez la fenetre de votre navigateur, l'image garde la taille initiale. Si vous souhaitez que votre application se redimensionne automatiquement, vous devez surveiller les changement de taille et modifier la camera et le renderer.

```
window.addEventListener("resize", () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
});
```

jsx

2D

Il est parfois utile d'afficher des elements 2D sur votre image, comme du texte ou des images, pour faire un **HUD** par exemple.

Pour cela, il est souvent plus simple d'utiliser directement du HTML/CSS superposé à votre `canvas`, mais certains outils existent quand meme en threejs.

Sprites

Les sprites sont des surfaces planes qui font **toujours** face à la caméra, il n'y a pas de projection 3D qui y sont appliqués.

```
const texture = new THREE.TextureLoader().load("/assets/textures/block/dirt.png");  
const material = new THREE.SpriteMaterial({ map: texture });  
const sprite = new THREE.Sprite(material);  
sprite.position.set(0, -5, 0);
```

Texte

<https://threejs.org/manual/?q=text#en/creating-text>

GUI

Vous pouvez utiliser des librairies qui vous permettent de modifier des paramètres de votre programme en temps réel:

<https://github.com/georgealways/lil-gui>

Physique

Three.js est un moteur de rendu, pas un moteur physique. Il ne gère pas nativement la gravité, les collisions ou les forces. Pour simuler de la physique, deux approches existent : la coder manuellement dans la boucle d'animation, ou déléguer à une librairie dédiée.

Simulation manuelle dans la boucle d'animation

Pour des effets simples comme la gravité, il suffit de mettre à jour la position d'un objet à chaque frame en appliquant une accélération.

L'idée : on maintient une variable `velocity` (vitesse verticale). À chaque frame, on lui soustrait la gravité multipliée par `delta`, puis on l'ajoute à la position Y de l'objet. Quand l'objet touche le sol ($Y \leq 0$), on arrête.

```
js
const GRAVITY = 9.8; // m/s2
let velocityY = 0; // Vitesse verticale initiale

function animate(lastRenderTime) {
  const delta = (Date.now() - lastRenderTime) / 1000;

  if (sphere.position.y > 0) {
    velocityY -= GRAVITY * delta; // Accélération vers le bas
    sphere.position.y += velocityY * delta; // Application de la vitesse
  }

  if (sphere.position.y < 0) {
    sphere.position.y = 0; // On s'arrête exactement au sol
    velocityY = 0;
  }
}

renderer.render(scene, camera);
}

renderer.setAnimationLoop(animate);
```

Cette approche fonctionne pour un cas simple, mais devient vite fastidieuse dès que l'on a plusieurs objets, des formes complexes ou des collisions entre objets.

Librairies de physique

Pour aller plus loin, il existe des moteurs physiques dédiés que l'on couple à Three.js. Le principe est toujours le même : le moteur physique calcule les positions et rotations, Three.js se charge uniquement du rendu. À chaque frame, on synchronise les positions des objets Three.js avec celles calculées par le moteur.

Cannon-es

<https://github.com/pmndrs/cannon-es>

Cannon-es est un fork maintenu de Cannon.js, écrit entièrement en JavaScript. C'est la librairie la plus simple à prendre en main pour débiter avec la physique dans Three.js.

Elle gère : gravité, forces, impulsions, collisions entre formes simples (box, sphere, plane...), friction et rebond.

```
npm install cannon-es
```

bash

```
import * as CANNON from "cannon-es";

// Monde physique
const world = new CANNON.World({ gravity: new CANNON.Vec3(0, -9.8, 0) });

// Corps physique (sphère)
const sphereBody = new CANNON.Body({
  mass: 1, // kg - 0 = statique
  shape: new CANNON.Sphere(0.5),
  position: new CANNON.Vec3(0, 10, 0),
});
world.addBody(sphereBody);

// Sol statique
const groundBody = new CANNON.Body({
  mass: 0,
  shape: new CANNON.Plane(),
});
groundBody.quaternion.setFromEuler(-Math.PI / 2, 0, 0);
world.addBody(groundBody);

// Boucle d'animation : on synchronise Three.js avec Cannon
function animate(lastRenderTime) {
  const delta = (Date.now() - lastRenderTime) / 1000;

  world.step(1 / 60, delta); // Avance la simulation

  // Copie position et rotation du corps physique vers le mesh Three.js
  sphere.position.copy(sphereBody.position);
  sphere.quaternion.copy(sphereBody.quaternion);

  renderer.render(scene, camera);
}
```

js

Pour appliquer des forces ou des impulsions :

```
// Force continue (vent, poussée...)
sphereBody.applyForce(new CANNON.Vec3(10, 0, 0), sphereBody.position);

// Impulsion instantanée (explosion, saut...)
sphereBody.applyImpulse(new CANNON.Vec3(0, 5, 0), sphereBody.position);
```

js

Rapier

<https://rapier.rs/>

Rapier est un moteur physique écrit en Rust et compilé en WebAssembly. Il est beaucoup plus performant que Cannon-es, notamment pour les scènes avec de nombreux objets. Il gère aussi bien la 2D que la 3D.

```
npm install @dimforge/rapier3d-compat
```

bash

```
import RAPIER from "@dimforge/rapier3d-compat";
await RAPIER.init(); // Initialisation du module WASM

const world = new RAPIER.World({ x: 0, y: -9.8, z: 0 });

// Corps rigide dynamique
const rigidBodyDesc = RAPIER.RigidBodyDesc.dynamic().setTranslation(0, 10, 0);
const rigidBody = world.createRigidBody(rigidBodyDesc);

// Collider (forme de collision)
const colliderDesc = RAPIER.ColliderDesc.ball(0.5);
world.createCollider(colliderDesc, rigidBody);

// Sol
const groundDesc = RAPIER.RigidBodyDesc.fixed();
const ground = world.createRigidBody(groundDesc);
world.createCollider(RAPIER.ColliderDesc.cuboid(10, 0.1, 10), ground);

function animate() {
  world.step();

  const pos = rigidBody.translation();
  sphere.position.set(pos.x, pos.y, pos.z);

  renderer.render(scene, camera);
}
```

js

Ammo.js

<https://github.com/kripken/ammo.js>

Ammo.js est le portage de **Bullet Physics** (moteur utilisé dans de nombreux jeux AAA) vers JavaScript via Emscripten. C'est le plus complet et le plus puissant des trois, mais aussi le plus complexe à utiliser.

Il supporte des formes de collision très avancées (convex hull, triangle mesh), les véhicules, les contraintes articulaires (ragdoll, portes...) et les corps mous (soft bodies).

```
// Initialisation du monde Bullet
const collisionConfiguration = new Ammo.btDefaultCollisionConfiguration();
const dispatcher = new Ammo.btCollisionDispatcher(collisionConfiguration);
```

js

```

const broadphase = new Ammo.btDbvtBroadphase();
const solver = new Ammo.btSequentialImpulseConstraintSolver();

const physicsWorld = new Ammo.btDiscreteDynamicsWorld(
    dispatcher, broadphase, solver, collisionConfiguration
);
physicsWorld.setGravity(new Ammo.btVector3(0, -9.8, 0));

// Boucle
function animate(lastRenderTime) {
    const delta = (Date.now() - lastRenderTime) / 1000;
    physicsWorld.stepSimulation(delta, 10);
    // ... synchronisation des meshes
}

```

Comparatif

Librairie	Langage	Performances	Complexité	Cas d'usage
Cannon-es	JavaScript	Moyenne	Faible	Prototypage, projets simples
Rapier	Rust/WASM	Élevée	Moyenne	Projets nécessitant de la performance
Ammo.js	C++/WASM	Élevée	Forte	Simulations avancées, jeux complexes

INFO

Pour la plupart des projets étudiants, **Cannon-es** est le meilleur point de départ : son API est intuitive et sa documentation accessible.

Glossaire

Vecteur

Un vecteur représente une direction et une magnitude dans l'espace. En 3D :

$$\vec{v} = (x, y, z)$$

La magnitude (longueur) du vecteur :

$$\|\vec{v}\| = \sqrt{x^2 + y^2 + z^2}$$

Un vecteur **normalisé** a une magnitude de 1 :

$$\hat{v} = \frac{\vec{v}}{\|\vec{v}\|}$$

Scalaire

Un scalaire est une valeur numérique simple, sans direction. Il peut multiplier un vecteur pour changer sa magnitude :

$$k \cdot \vec{v} = (k \cdot x, k \cdot y, k \cdot z)$$

Matrice

Une matrice 4x4 est utilisée en 3D pour représenter des transformations (translation, rotation, échelle). En Three.js, [Matrix4](#) est omniprésente :

$$M = \begin{pmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{pmatrix}$$

Angles d'Euler

Les angles d'Euler décomposent une rotation 3D en trois rotations successives autour des axes **X**, **Y**, **Z** (respectivement **pitch**, **yaw**, **roll**) :

$$R = R_Z(\psi) \cdot R_Y(\theta) \cdot R_X(\phi)$$

avec $\phi \in [-180^\circ, 180^\circ]$, $\theta \in [-90^\circ, 90^\circ]$, $\psi \in [-180^\circ, 180^\circ]$.

C'est la représentation la plus intuitive, mais l'ordre des rotations **compte** (

$R_X \cdot R_Y \neq R_Y \cdot R_X$), et elle souffre du **gimbal lock** : lorsque $\theta = \pm 90^\circ$, les axes **X** et **Z** s'alignent et on perd un degré de liberté.

Quaternion

Un quaternion représente une **rotation** dans l'espace 3D sans souffrir du blocage de cardan (*gimbal lock*). Il est composé d'une partie scalaire et d'un vecteur :

$$q = (w, x, y, z) \quad \text{avec} \quad w^2 + x^2 + y^2 + z^2 = 1$$

Une rotation d'angle θ autour d'un axe unitaire $\hat{u} = (u_x, u_y, u_z)$ s'encode :

$$q = \left(\cos \frac{\theta}{2}, u_x \sin \frac{\theta}{2}, u_y \sin \frac{\theta}{2}, u_z \sin \frac{\theta}{2} \right)$$

Normale

Un vecteur normal est un vecteur **perpendiculaire** à une surface en un point donné. Il est unitaire par convention :

$$\hat{n} = \frac{\vec{a} \times \vec{b}}{\|\vec{a} \times \vec{b}\|}$$

où $\vec{a}$ et $\vec{b}$ sont deux arêtes du triangle. Les normales sont indispensables pour le calcul de l'éclairage et les réponses aux collisions.

Rayon

Un rayon est une demi-droite définie par une origine $\vec{o}$ et une direction $\hat{d}$ (unitaire). Tout point du rayon s'écrit :

$$\vec{r}(t) = \vec{o} + t \cdot \hat{d} \quad (t \geq 0)$$

Utilisé pour le **raycasting** : détecter ce que vise le curseur, les tirs, la ligne de vue.

AABB

Une *Axis-Aligned Bounding Box* est une boîte de collision dont les faces sont parallèles aux axes du repère. Elle se définit par deux points :

$$\text{AABB} = (\vec{p}_{min}, \vec{p}_{max}) \quad \text{avec} \quad \vec{p}_{min} = (x_{min}, y_{min}, z_{min})$$

Un point $\vec{p}$ est à l'intérieur si :

$$x_{min} \leq p_x \leq x_{max} \quad \wedge \quad y_{min} \leq p_y \leq y_{max} \quad \wedge \quad z_{min} \leq p_z \leq z_{max}$$

UV

Les coordonnées UV sont des coordonnées 2D $(u, v) \in [0, 1]^2$ qui associent chaque sommet d'un maillage à un point d'une texture :

$$\text{couleur}(u, v) = \text{texture}[u \cdot W, v \cdot H]$$

où W et H sont les dimensions en pixels de la texture. u correspond à l'axe horizontal, v à l'axe vertical.

Opérations

Vectorielle

Dot product

Le produit scalaire (dot product) de deux vecteurs donne un **scalaire**. Il mesure à quel point deux vecteurs sont alignés :

$$\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y + a_z b_z = \|\vec{a}\| \|\vec{b}\| \cos \theta$$

- Si $\vec{a} \cdot \vec{b} = 0$: les vecteurs sont **perpendiculaires**
- Si $\vec{a} \cdot \vec{b} > 0$: ils pointent dans le même sens
- Si $\vec{a} \cdot \vec{b} < 0$: ils pointent en sens opposé

Cross product

Le produit vectoriel (cross product) de deux vecteurs donne un **vecteur** perpendiculaire aux deux :

$$\vec{a} \times \vec{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$$

Sa magnitude vaut $\|\vec{a}\| \|\vec{b}\| \sin \theta$. Utilisé notamment pour calculer les **normales** de surfaces.

Matricielle

Multiplication matricielle

Appliquer une transformation **B** puis **A** à un vecteur $\vec{v}$:

$$\vec{v}' = A \cdot B \cdot \vec{v}$$

L'ordre compte : $A \cdot B \neq B \cdot A$ en général.

Pour transformer un point, on multiplie la matrice par le vecteur colonne. En dimension **n** :

$$\begin{pmatrix} x'_1 \\ x'_2 \\ \vdots \\ x'_n \end{pmatrix} = \begin{pmatrix} m_{11} & m_{12} & \cdots & m_{1n} \\ m_{21} & m_{22} & \cdots & m_{2n} \\ \vdots & & \ddots & \vdots \\ m_{n1} & m_{n2} & \cdots & m_{nn} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

Chaque composante du résultat est la somme des produits de la ligne **i** de **M** avec le vecteur :

$$x'_i = m_{i1} \cdot x_1 + m_{i2} \cdot x_2 + \cdots + m_{in} \cdot x_n = \sum_{j=1}^n m_{ij} \cdot x_j$$

En 3D homogène (**n = 4**, avec $x_4 = 1$) :

$$\begin{pmatrix} x' \\ y' \\ z' \\ 1 \end{pmatrix} = M \cdot \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix}$$

$$x' = m_{11}x + m_{12}y + m_{13}z + m_{14}$$

$$y' = m_{21}x + m_{22}y + m_{23}z + m_{24}$$

$$z' = m_{31}x + m_{32}y + m_{33}z + m_{34}$$

Transformation

Déplacer, tourner ou redimensionner un objet revient à lui appliquer une matrice. On distingue trois transformations fondamentales :

Translation d'un vecteur $\vec{t} = (t_x, t_y, t_z)$:

$$T = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Mise à l'échelle par facteurs (s_x, s_y, s_z) :

$$S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation d'angle θ autour de l'axe Z :

$$R_z = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

La transformation complète d'un objet (modèle → monde) combine ces matrices :

$$M_{model} = T \cdot R \cdot S$$

Caméra

Projection perspective

La projection perspective transforme les coordonnées 3D en coordonnées 2D écran en simulant la perception humaine : les objets lointains paraissent plus petits. Elle est définie par :

- **FOV** (*Field of View*) : angle d'ouverture vertical α
- **Aspect ratio** : rapport largeur/hauteur de l'écran $a = W/H$
- **Near/Far** : distances minimale n et maximale f du volume visible

Un point 3D (x, y, z) est projeté en :

$$x_{\text{écran}} = \frac{x}{z \cdot \tan(\alpha/2)} \quad y_{\text{écran}} = \frac{y}{z \cdot a \cdot \tan(\alpha/2)}$$

Physique

Forces

La force appliquée à un objet de masse m produit une accélération $\vec{a}$:

$$\vec{F} = m \cdot \vec{a}$$

Plusieurs forces peuvent s'additionner (gravité, vent, collisions...) :

$$\vec{F}_{total} = \vec{F}_1 + \vec{F}_2 + \dots + \vec{F}_n$$

Position, vitesse, accélération

Ces trois grandeurs sont liées par la dérivation / intégration par rapport au temps t :

$$\vec{a}(t) = \frac{d\vec{v}}{dt}$$

$$\vec{v}(t) = \frac{d\vec{p}}{dt}$$

$$\vec{a}(t) = \frac{d^2\vec{p}}{dt^2}$$

$$\vec{v}(t) = \int \vec{a} dt \quad \vec{p}(t) = \int \vec{v} dt$$

En simulation temps réel, on approxime avec un pas de temps Δt (méthode d'Euler) :

$$\vec{v}_{n+1} = \vec{v}_n + \vec{a}_n \cdot \Delta t$$

$$\vec{p}_{n+1} = \vec{p}_n + \vec{v}_{n+1} \cdot \Delta t$$

Gravité et animation

Sous l'effet de la gravité seule ($g \approx 9.8 \text{ m/s}^2$), la hauteur à l'instant t est :

$$p_y(t) = p_{y_0} + v_{y_0} \cdot t - \frac{1}{2}g \cdot t^2$$

En animation image par image, on met à jour la hauteur à chaque pas Δt :

$$v_{y_{n+1}} = v_{y_n} - g \cdot \Delta t$$

$$p_{y_{n+1}} = p_{y_n} + v_{y_{n+1}} \cdot \Delta t$$

Exercices pratiques

- Ajouter différents objets à la scène (sphere, cube, ...)
- Changer la couleur de ces objets
- Faire déplacer ces objets tout seul
- Créer un groupe de plusieurs objets et déplacer tous les objets du groupe en une seule fois
- Faire en sorte de pouvoir déplacer un objet avec le clavier
- Télécharger un modèle 3D sur internet et l'insérer dans la scène
- Ajouter une texture à un cube
- Appliquer un système de gravité et faire tomber des objets sur un sol
- Détecter quand deux objets rentrent en collision, et émettre un son lorsque cela se produit
- Tester différents modes de contrôles (Pointer, Orbit)
- Détecter quel objet est pointé par l'utilisateur (selon le mode de contrôle)